

Implementation of Backpropagation Neural Network:

```
import numpy as np
```

Dense Layer:

```
class Dense:
    def __init__(self, input_size, output_size):
        limit = np.sqrt(6 / (input_size + output_size))
        self.weight = np.random.uniform(
            -limit,
            limit,
            (input_size, output_size)
        )
        self.bias = np.zeros((1, output_size))
    def forward(self, x):
        self.x = x
        self.z = (np.dot(self.x, self.weight) + self.bias)
        return self.z
    def backward(self, delta, lr):
        DW = np.dot(self.x.T, delta)
        DB = np.sum(delta, axis=0, keepdims=True)
        grad = np.dot(delta, self.weight.T)
        self.weight -= lr * DW
        self.bias -= lr * DB
        return grad
```

Bipolar Sigmoid Activation Function Class:

```
class Bipolar_Sigmoid:
    def forward(self, z):
        self.z = z
        self.y_hat = (2 / (1 + np.exp(-z))) - 1
        return self.y_hat
    def backward(self):
        self.da = (0.5) * (1 - self.y_hat ** 2)
        return self.da
```

Least Square Error Class:

```
class Least_square_error:
    def forward(self, y, y_hat):
        self.y = y
        self.y_hat = y_hat
        self.l = np.mean((0.5) * (y - y_hat) ** 2)
        return self.l
    def backward(self):
        self.dl = self.y_hat - self.y
        return self.dl
```

Trainer Class:

```
class Trainer:

    def __init__(self, model, x, y, epochs, lr):
        self.model = model

        self.x = x

        self.y = y

        self.epochs = epochs

        self.lr = lr

        self.loss_fn = Least_Squared_Error()

    def train(self):
        for epoch in range(self.epochs):
            z = self.x

            for layer in self.model:
                z = layer.forward(z)

            loss = self.loss_fn.forward(self.y, z)

            delta = self.loss_fn.backward()

            for layer in reversed(self.model):
                if hasattr(layer, "weight"):
                    delta = layer.backward(delta, self.lr)
                else:
                    delta = delta * layer.backward()

            print(f"Epoch: {epoch+1} | Loss: {loss:5.2f} | Learning
Rate: {self.lr}")
```

Defining the Model:

```
model = [  
    Dense(3, 2),  
    Bipolar_Sigmoid(),  
    Dense(2, 1),  
    Bipolar_Sigmoid()  
]
```

Defining the Weights and bias:

```
model[0].weight = np.array([  
    [0.2, 0.1],  
    [0.2, 0.1],  
    [0.2, 0.1]  
])  
model[0].bias = np.array([  
    [0.1, 0.1]  
])  
model[2].weight = np.array([  
    [0.1],  
    [0.1]  
])  
model[2].bias = np.array([  
    [0.1]  
])
```

Inputs and Targers:

```
x = np.array([[1, 1, 1]])  
y = np.array([[1]])
```

Weights and Bias Before Training:

```
print(f"""Model Weigts and Bias:\nLayer 1:\nWeight:\n {model[0].weight}\nBias: {model[0].bias}\nLayer 2:\nWeight:\n {model[2].weight}\n Bias: {model[2].bias}""")Predicted\noutput: {y_hat}""")
```

Model Weigts and Bias:

Layer 1:

Weight:

[[0.2 0.1]

[0.2 0.1]

[0.2 0.1]]

Bias: [[0.1 0.1]]

Layer 2:

Weight:

[[0.1]

[0.1]]

Bias: [[0.1]]

Hyper-Parameters:

```
EPOCHS = 1
```

```
LR = 0.1
```

Creating Trainer Object and Model Training:

```
trainer = Trainer(\n    model = model,\n    x = x,\n    y = y,\n    epochs = EPOCHS,\n    lr = LR\n)\ntrainer.train()
```

Epoch: 1 | Loss: 0.43 | Learning Rate: 0.1

Weights and Bias After Training:

```
print(f"""Model Weights and Bias:\nLayer 1:  
Weight:\n {model[0].weight}\nBias: {model[0].bias}\n  
Layer 2:  
Weight:\n {model[2].weight}\n Bias: {model[2].bias}""")
```

```
Model Weights and Bias:  
Layer 1:  
Weight:  
[[0.20203544 0.10220572]  
 [0.20203544 0.10220572]  
 [0.20203544 0.10220572]]  
Bias: [[0.10203544 0.10220572]]  
  
Layer 2:  
Weight:  
[[0.11544053]  
 [0.10906005]]  
Bias: [[0.14590264]]
```

Prediction After Training:

```
z = x  
for layer in model:  
    z = layer.forward(z)  
prediction = z[0][0]  
if prediction > 0:  
    print(1)  
else:  
    print(-1)
```